Assessment

➢ DB and Python Framework

Theme: Social Media Profile Manager

Section A: Conceptual Understanding

1. Explain the Django Request-Response Cycle and how it differs from a standard Python script execution.

- Django Request-Response Cycle:
  1. A user sends a request through a web browser.
  2. Django receives the request through the URL dispatcher.
  3. The matching View function/class processes the request.
  4. The View may interact with Models (database) and Templates.
  5. Django generates a response and sends it back to the browser.
- Standard Python Script Execution:
  - A Python script runs from top to bottom once and then stops.
  - It does not wait for web requests or generate HTTP responses.
- Difference:
  Django is event-driven and handles multiple web requests continuously, whereas a standard Python script executes sequentially and terminates after completion.

2. Explain why Django Model Fields (CharField, IntegerField) are more robust for profile data than Python dynamic typing.

Django Model Fields enforce data types and validation rules at the database level.

Example:

```
class Profile(models.Model):
  username = models.CharField(max_length=50)
  age = models.IntegerField()
```

- **Advantages:**
  - Prevents invalid data storage.
  - Provides automatic validation.
  - Creates proper database columns.
  - Improves data consistency.

  In normal Python, variables can change types anytime:

  ```
  age = 25
  age = "twenty five"
  ```

  This flexibility can cause errors, while Django fields maintain strict data integrity.

3. Explain how Django Forms handle automated input validation for usernames and age ranges.

Django Forms automatically validate user input before processing.

Example:

```
from django import forms

class UserForm(forms.Form):
  username = forms.CharField(max_length=20)
  age = forms.IntegerField(min_value=18, max_value=60)
```

- **Validation Performed:**
  - Username cannot exceed 20 characters.
  - Age must be an integer.

- Age must be between 18 and 60.
- Required fields cannot be left empty.

When invalid data is submitted, Django displays error messages automatically without writing extra validation code.

4. Explain how to implement conditional logic in Django Templates to toggle account visibility.

Django Templates use {% if %} statements for conditional rendering.

Example:

```
{% if user.is_public %}
  <p>Profile is Visible</p>
{% else %}
  <p>Profile is Hidden</p>
{% endif %}
```

- **How it works:**
  - If is_public is True, the profile is shown.
  - If is_public is False, the profile is hidden.

This allows dynamic content display based on user settings or database values.

5. Explain the difference between iterating through a Python list and a Django QuerySet.

- **Python List:**

```
users = ["Ayu", "Raj", "Priya"]

for user in users:
  print(user)
```

- Data is already loaded into memory.
- Fast for small datasets.
- Cannot directly communicate with a database.

➢ **Django QuerySet:**

```
users = User.objects.all()

for user in users:
    print(user.username)
```

- Represents database records.
- Data is fetched from the database when needed (lazy loading).
- Supports filtering, sorting, and database operations.

➢ **Difference:**
A list stores data in memory, while a QuerySet retrieves and manages data from the database efficiently.

6. Explain why the Django ORM is preferred over Python dictionaries for persistent profile storage.

➢ **Python Dictionary:**

```
profile = {
    "username": "Ayu",
    "age": 22
}
```

➢ **Limitations:**

- Data is lost when the program stops.
- No database storage.
- Difficult to manage large datasets.

- **Django ORM Example:**

```
Profile.objects.create(
  username="Ayu",
  age=22
)
```

- **Advantages of ORM:**
  - Stores data permanently in a database.
  - Supports searching, updating, and deleting records.
  - Provides security against SQL injection.
  - Handles relationships between models.
  - Scales efficiently for large applications.